How to use viem with Hardhat

viem is a modern, type-safe library to deploy contracts, manage accounts, read chain state, and more. It's the library we recommend for interacting with Ethereum. You can integrate viem with Hardhat by using the `hardhat-viem` plugin.

#Setup

If you have already initialized a viem-based project using `hardhat --init`, you don't need to do anything else.

If you want to add the plugin manually:

1. Install the plugin:

```
npm install --save-dev @nomicfoundation/hardhat-viem

# or
pnpm install --save-dev @nomicfoundation/hardhat-viem

# or
yarn add --dev @nomicfoundation/hardhat-viem
```

2.
3. Add it to the list of plugins in your Hardhat configuration:

```
import hardhatViem from "@nomicfoundation/hardhat-viem";

const config: HardhatUserConfig = {
  plugins: [
    hardhatViem,
    // ...other plugins...
  ],
  // ...other config...
};

export default config;
```

4.

#Connecting to networks

In Hardhat, you interact with networks using *network connections*. You can create connections with the network manager, which can be imported directly from Hardhat:

```
import { network } from "hardhat";

const connection = await network.connect();
```

Plugins can extend the objects returned by the network manager. The `hardhat-viem` plugin extends them with a `viem` property, which provides helpers to interact with the network you connected to:

```
const { viem } = await network.connect();

const publicClient = await viem.getPublicClient();
console.log("Latest block number:", await publicClient.getBlockNumber());
```

#Using viem clients

Viem functionality is grouped into clients. The `hardhat-viem` plugin helps you create viem clients more easily.

You can create a public client using the `getPublicClient` method:

```
const { viem } = await network.connect();

const publicClient = await viem.getPublicClient();

console.log("Latest block number:", await publicClient.getBlockNumber());
```

Use the `getWalletClients` function to obtain wallet clients. It returns an array of wallet clients, one for each account set up in the Hardhat config:

```
const [senderClient, receiverClient] = await viem.getWalletClients();

await senderClient.sendTransaction({
  to: receiverClient.account.address,
  value: 10n ** 18n,
});
```

Finally, if you are connecting to a Hardhat network, you can call `getTestClient` to get a test client:

```
const testClient = await viem.getTestClient();

await testClient.mine({
  blocks: 10,
});
```

#Deploying and interacting with contracts

`hardhat-viem` includes a `deployContract` function to deploy contracts defined in the project. This function returns a viem contract instance of the deployed contract:

```
import { network } from "hardhat";

const { viem } = await network.connect();
const counter = await viem.deployContract("Counter");

await counter.write.inc();

console.log("Counter value:", await counter.read.x());
```

If the constructor takes parameters, you can pass them as the second argument:

```
const initialValue = 10n;
const counter = await viem.deployContract("Counter", [initialValue]);
```

By default, contracts are deployed from the first account defined in the Hardhat configuration, but you can specify a different one:

```
const [wallet1, wallet2] = await viem.getWalletClients();
```

```
const counter = await viem.deployContract("Counter", [10n], {
  client: {
    wallet: wallet2,
  },
});
```

The `deployContract` function waits until the contract is deployed. If you just want to send the deployment without waiting until it's mined, you can use `sendDeploymentTransaction`:

```
const { deploymentTransaction } = await viem.sendDeploymentTransaction(
  "Counter",
  [10n],
  {
    client: {
      wallet: wallet2,
    },
  }
);
```

All the previous examples deploy a new contract instance, but sometimes you need to interact with an already deployed contract. In those cases, use the `getContractAt` function:

```
const counterAddress = "0x1234567890123456789012345678901234567890";
const counter = await viem.getContractAt("Counter", counterAddress);
```

#Using contracts from an npm dependency

You can also use a contract defined in an npm dependency with `hardhat-viem`.

To do this, configure Hardhat to compile the contract and generate its artifacts. Read this guide to learn how.

This lets you to use the `hardhat-viem` helpers to interact with it, just like any other contract in your project.

#Viem assertions

The example project includes the `hardhat-viem-assertions` plugin, which helps you write expressive TypeScript tests in viem-based projects.

This plugin adds an `assertions` property to the `viem` object, giving you utility functions to test contract behavior more easily. For example, the following test checks that calling a function emits the expected event:

```
it("Should emit the Increment event when calling the inc() function", async function () {
  const counter = await viem.deployContract("Counter");

  await viem.assertions.emitWithArgs(
    counter.write.inc(),
    counter,
    "Increment",
    [1n]
  );
});
```

You can also assert that transactions revert (and why), or check that account balances change as expected.

#Type-safe contract interactions

Viem has powerful typing capabilities, triggering compilation errors when you make mistakes like using the wrong type in a function argument or sending value to a non-payable function:

```
// doesn't compile if getItem expects a number but receives a string:
await contract.read.getItem(["3"]);

// doesn't compile if setItem is not payable:
await contract.write.setItem([3, "three"], {
  value: 1000n,
});
```

When using viem on its own, you need to explicitly use the contract's ABI to get properly inferred types. The `hardhat-viem` plugin handles this automatically in helpers like `deployContract` or `getContractAt`.

#Troubleshooting contract type errors

Contract types are updated when the project is compiled. If you are getting a compilation error that you don't expect, make sure you've run `hardhat build`.

Note that VSCode may not always pick up the type updates automatically. If you are still getting unexpected TypeScript errors after compiling the project, open the Command Palette and run `TypeScript: Reload Project`.

#Using viem as a module

The `viem` object in the connection only includes the functionality added by `hardhat-viem`. To use viem's own functionality, import it from the `viem` module:

```
import { keccak256 } from "viem";
import { network } from "hardhat";

const { viem } = await network.connect();
```

Keep in mind that you can get a name clash if you use a namespace import:

```
import * as viem from "viem";
import { network } from "hardhat";

// this is an error because viem is already declared
const { viem } = await network.connect();
```

One way to work around this problem is to use a different name for the Hardhat viem object:

```
const { viem: hhViem } = await network.connect();
```

```
const publicClient = await hhViem.getPublicClient();
```